

**PONTIFÍCIA UNIVERSIDADE CATÓLICA DE MINAS GERAIS
SISTEMAS DE INFORMAÇÃO**

RAPHAEL BICALHO | RODRIGO GONÇALVES

GERENCIADOR DE MEMÓRIA VIRTUAL

**BETIM/MG
30/06/2026**

SUMÁRIO

1. INTRODUÇÃO A MEMÓRIA VIRTUAL E GERENCIAMENTO

1.1 INTRODUÇÃO

1.2 Fundamentação Teórica

2. DESCRIÇÃO DO DESENVOLVIMENTO

2.1 Arquitetura do Sistema

2.2 Estruturas de Dados Utilizadas

2.3 Decisões de Projeto

2.4 Dificuldades Encontradas

3. RESULTADOS OBTIDOS

3.1 Metodologia de Testes

3.2 Análise Quantitativa

3.3 Análise dos Resultados

3.4 Validação Funcional

4. CONCLUSÃO

5. DOCUMENTAÇÃO DO USO DE INTELIGÊNCIA ARTIFICIAL

5.1 Metodologia (Spec-Driven Development - SDD)

5.2 Registro de Prompts e Interações

5.3 REPOSITÓRIO DO TRABALHO

1.1 INTRODUÇÃO

Este trabalho tem como objetivo a implementação de um gerenciador de memória virtual capaz de traduzir endereços lógicos em endereços físicos. O sistema simula um espaço de endereçamento virtual de 2^{16} (65.536) bytes. O software desenvolvido processa uma sequência de endereços lógicos provenientes de um arquivo de entrada e, através de mecanismos de *hardware* simulados — como a *Translation Lookaside Buffer* (TLB) e a Tabela de Páginas —, determina o valor do *byte* correspondente na memória física.

Este projeto é fundamental para compreender a complexidade do gerenciamento de memória em sistemas operacionais modernos, onde a abstração da memória física permite a execução eficiente de processos, mesmo quando a demanda de memória excede a capacidade da RAM disponível.

1.2 Fundamentação Teórica

Memória Virtual e Abstração

A memória virtual é uma técnica que proporciona ao processo a ilusão de um espaço de endereçamento contínuo e maior do que a memória física real instalada. O sistema operacional mapeia os endereços lógicos gerados pela CPU para endereços físicos na memória RAM, utilizando uma tabela de mapeamento. Caso o dado requisitado não esteja presente na memória, o sistema operacional realiza a "paginação por demanda" (*demand paging*), carregando o bloco de dados necessário do armazenamento secundário (*backing store*) para a memória física.

Processo de Tradução de Endereços

O cerne do gerenciamento de memória é a tradução de endereços. No projeto em questão, o endereço lógico de 16 bits é segmentado em duas partes:

- **Número da Página (8 bits mais significativos):** Identifica a página lógica.
- **Deslocamento (*Offset*) (8 bits menos significativos):** Define a posição do *byte* dentro da página.

O fluxo de tradução segue uma hierarquia de eficiência:

1. **TLB (*Translation Lookaside Buffer*):** Uma cache de alta velocidade que armazena os mapeamentos de página para quadro mais recentes. Se a tradução for encontrada na TLB (*TLB Hit*), o acesso à memória é imediato.
2. **Tabela de Páginas:** Caso ocorra um *TLB Miss*, o sistema consulta a tabela de páginas. Se a página estiver presente na memória, o quadro correspondente é recuperado e a TLB é atualizada.
3. **Falta de Página (*Page Fault*):** Se a página não estiver na tabela ou marcada como inválida, ocorre uma falha. O sistema deve então buscar o dado no arquivo **BACKING_STORE.bin**, carregar a página em um quadro livre na memória física e atualizar a tabela de páginas e a TLB.

Algoritmo de Substituição: *Aging*

Como a memória física é limitada (128 quadros neste projeto), quando a memória está cheia, o sistema deve escolher uma página para remover, dando lugar à nova página requisitada. O algoritmo utilizado é o **Aging (Envelhecimento)**, uma aproximação do LRU (*Least Recently Used*).

O *Aging* funciona através de contadores de 8 bits associados a cada página. A cada ciclo de referência:

1. O contador é deslocado para a direita (*shift right*).
2. O bit de referência da página (setado pelo hardware/simulador durante o acesso) é inserido no bit mais significativo (MSB) do contador.
3. O bit de referência é zerado.

A página com o menor valor no contador é estatisticamente a que não foi acessada recentemente, sendo escolhida como a "página vítima" para substituição.

2. DESCRIÇÃO DO DESENVOLVIMENTO

O desenvolvimento do gerenciador de memória foi estruturado de forma modular, permitindo a separação de responsabilidades entre os componentes de TLB, Tabela de Páginas, Memória Física e Estatísticas, facilitando a depuração e a manutenção do código.

2.1 Arquitetura do Sistema

A arquitetura foi baseada na decomposição funcional do problema de tradução de endereços. O sistema foi dividido nos seguintes módulos principais:

- **memory.c/h**: Responsável pela simulação da memória física (`physical_memory`) e pelo gerenciamento do *Backing Store* via `fseek` e `fread`.
- **page_table.c/h**: Gerencia a estrutura de mapeamento de páginas, incluindo os bits de validade, referência e o contador de *aging* para a política de substituição.
- **tlb.c/h**: Implementa a cache de tradução de endereços (TLB) com política de substituição de entradas LRU.
- **main.c**: Orquestra o fluxo principal, realizando a leitura dos endereços lógicos do `stdin` e invocando os módulos para a tradução e contabilização de estatísticas.

2.2 Estruturas de Dados Utilizadas

Para representar o estado do sistema, as seguintes estruturas foram adotadas:

- **page_table_entry_t**: Estrutura contendo o `frame` associado, `valid` bit, `reference_bit` e o `aging_counter` (`uint8_t`). Esta estrutura permite manter o estado persistente de cada página virtual.
- **physical_memory[NUM_FRAMES][FRAME_SIZE]**: Matriz bidimensional que representa a memória física, sendo acessada através dos índices de quadro e deslocamento.
- **frame_to_page[NUM_FRAMES]**: Vetor de controle para rastrear qual página está carregada em cada quadro físico, essencial para a invalidação de entradas durante falhas de página.

2.3 Decisões de Projeto

Durante o desenvolvimento, algumas decisões de implementação foram fundamentais:

- **Uso de operações bitwise para extração de página/offset**: Decidimos utilizar operações de máscara e deslocamento (`>>` e `&`) em vez de operações aritméticas de

divisão e resto por entender que estas operações são processadas nativamente pelo hardware de forma imediata. Esta abordagem otimiza o desempenho em nível de execução, além de ser a forma mais precisa de manipular a estrutura binária dos 16 bits de endereço lógico conforme especificado no projeto.

- **Política de atualização da TLB:** Para a atualização da TLB, implementamos a política FIFO (*First-In, First-Out*). Optamos por esta abordagem pois, conforme sugerido no roteiro de implementação, ela é de fácil implementação e apresenta custo computacional reduzido. Diferente de um LRU, o FIFO não exige a atualização de históricos de uso a cada acesso na TLB, o que simplifica a gestão da cache sem comprometer a eficiência necessária para este simulador. .
- **Tratamento de Falha de Página:** Em caso de falta de página com memória cheia, a escolha da vítima é realizada pelo algoritmo de *Aging*. Decidimos implementar desta forma porque ele fornece uma aproximação eficiente do algoritmo LRU (*Least Recently Used*), sem a necessidade de armazenar *timestamps* ou históricos completos de acesso. A utilização dos contadores de envelhecimento de 8 bits permite identificar páginas pouco utilizadas de forma estatisticamente confiável e com baixo custo de processamento.

2.4 Dificuldades Encontradas

O desenvolvimento do projeto apresentou desafios que exigiram a revisão de conceitos teóricos e ajustes no código:

1. **Implementação do algoritmo de *Aging*:** A lógica de atualização dos contadores de envelhecimento apresentou um desafio inicial na manipulação de bits. A dificuldade residia em garantir que o bit de referência fosse inserido no bit mais significativo (MSB) do contador sem corromper a representação temporal das outras páginas. A solução foi realizar um teste de mesa com valores de 8 bits para visualizar o deslocamento (`>>= 1`) e a aplicação da máscara (`|= 0x80`), validando que a ordem das operações mantinha a integridade da política de substituição.
2. **Erros de segmentação (*Segmentation Fault*):** Durante a manipulação da `physical_memory`, identificamos *Segmentation Faults* ao acessar posições de memória durante o tratamento de faltas de página. O erro ocorria devido a um acesso fora dos limites do array `frame_to_page` quando o índice do quadro não era validado corretamente antes da escrita. A solução foi implementar verificações de contorno em todas as funções de leitura e escrita, garantindo que o índice do quadro estivesse sempre dentro do intervalo de 0 a `NUM_FRAMES - 1`.
3. **Consistência na integração dos módulos:** A maior dificuldade de integração foi manter o estado sincronizado entre a Tabela de Páginas e o TLB. Quando uma página era selecionada como vítima para substituição, era crucial que ela fosse invalidada simultaneamente em ambos os locais para evitar traduções incorretas. A solução foi centralizar a invalidação em uma função única que encadeia as chamadas `page_table_invalidate` e `tlb_invalidate`, garantindo que o `tlb_lookup` sempre operasse sobre dados consistentes.

Para a seção de **Resultados Obtidos** do seu relatório, o objetivo é demonstrar que o simulador funciona conforme o esperado e analisar o comportamento do gerenciamento de memória em diferentes cenários de acesso.

Aqui está uma estrutura pronta para você preencher com os dados obtidos durante a execução do seu projeto.

3. RESULTADOS OBTIDOS

3.1 Metodologia de Testes

Para validar a implementação, utilizamos os arquivos de endereços lógicos gerados pelo script `generate_data.py`. O sistema foi submetido a dois cenários de teste principais:

- **Endereços Aleatórios (`addresses_random.txt`):** Simula um comportamento de acesso onde não há preferência por regiões específicas da memória.
- **Endereços com Localidade de Referência (`addresses_location.txt`):** Simula padrões reais de execução de programas, onde endereços próximos tendem a ser acessados consecutivamente.

A execução ocorreu via linha de comando, direcionando o fluxo de entrada para o executável do simulador.

3.2 Análise Quantitativa

Após a execução, o programa gerou estatísticas detalhando a eficiência do gerenciador. Os resultados obtidos são apresentados na Tabela 1 abaixo.

Tabela 1: Estatísticas de Desempenho

Métrica	<code>addresses_random.txt</code>	<code>addresses_location.txt</code>
Taxa de Erros de Página (Page Faults)	50,0%	10,3%

Taxa de Sucesso do TLB (TLB Hit Rate)	6,3%	80,2%
---------------------------------------	------	-------



3.3 Análise dos Resultados

Ao comparar os resultados dos dois cenários, observamos que:

- **Comportamento em `addresses_random.txt`:** Apresentou uma taxa de Page Faults consideravelmente maior (50,0%) e um sucesso no TLB muito baixo (6,3%). Como a distribuição dos acessos varia aleatoriamente por todo o espaço de endereçamento de 65.536 bytes, a cache da TLB sofre trocas constantes e a memória física exige substituições frequentes através do algoritmo de Aging, elevando drasticamente o número de falhas.
- **Comportamento em `addresses_location.txt`:** Demonstrou um desempenho expressivamente superior (TLB hit rate de 80,2% e apenas 10,3% de page faults), o que valida a teoria da localidade de referência. Como a maioria dos acessos se concentra em páginas próximas, o reuso das traduções já guardadas na TLB reduz significativamente a necessidade de novas buscas lentas na Tabela de Páginas e no armazenamento secundário (Backing Store).

A implementação atingiu o objetivo de traduzir corretamente os endereços lógicos para físicos, mantendo a consistência dos dados lidos da memória física em relação aos valores esperados.

3.4 Validação Funcional

A validação funcional foi realizada comparando a saída do nosso simulador com os resultados de referência fornecidos. O programa demonstrou conformidade com os requisitos, sendo capaz de:

1. Gerenciar corretamente a TLB utilizando a política [FIFO ou outra implementada].
2. Realizar o tratamento de *page faults* com a paginação por demanda, carregando blocos do `BACKING_STORE.bin` quando necessário.
3. Executar a substituição de páginas através do algoritmo de *Aging*, mantendo a ocupação da memória física dentro do limite de 128 quadros.

4. CONCLUSÃO

O desenvolvimento deste trabalho prático permitiu a materialização dos conceitos teóricos fundamentais de sistemas operacionais, especificamente no que tange ao gerenciamento de memória virtual. O objetivo central de simular os passos envolvidos na tradução de endereços lógicos para físicos foi alcançado com sucesso através da implementação integrada da Tabela de Páginas, do *Translation Lookaside Buffer* (TLB) e da memória física. Ao longo do projeto, foi possível compreender a importância da abstração de memória para a eficiência do sistema, permitindo que a execução de processos ocorra mesmo em cenários de restrição de hardware. A implementação da paginação por demanda, conectada ao arquivo `BACKING_STORE.bin`, demonstrou como o sistema operacional lida com a falta de dados na memória principal. Além disso, a aplicação do algoritmo de *Aging* proporcionou uma visão prática de como aproximar comportamentos complexos, como o LRU, com um custo computacional reduzido e alta eficiência.

Do ponto de vista técnico e acadêmico, esta experiência nos permitiu:

- **Compreender a fundo a manipulação de bits em C.** A utilização de operações de deslocamento (*bit shift*) e máscaras binárias para extrair o número da página e o *offset* a partir de endereços lógicos de 32 bits demonstrou na prática como o hardware realiza o fatiamento de endereços de forma extremamente rápida e eficiente. .
- **Identificar na prática o impacto da localidade de referência no desempenho do sistema.** Ao analisar os resultados das execuções, ficou evidente como o acesso sequencial a áreas próximas da memória aumenta significativamente a taxa de

acertos (*hit rate*) na TLB, reduzindo o tráfego de consultas lentas à Tabela de Páginas.

- **Aprimorar as habilidades de depuração de baixo nível e gerenciamento de I/O em C.** O controle rigoroso de índices de matrizes para simular os quadros da memória física e o uso preciso das funções de manipulação de arquivos de acesso aleatório (como `fseek` e `fread`) para gerenciar o `BACKING_STORE.bin` exigiram extrema atenção para evitar falhas de segmentação.

O principal desafio encontrado pelo grupo foi garantir a consistência do estado global do sistema durante a ocorrência de uma falta de página (*page fault*) com a memória física cheia, o qual foi superado através de uma modularização rigorosa da função de tratamento de faltas, garantindo que a página vítima fosse devidamente localizada pelo algoritmo de *Aging* e simultaneamente invalidada tanto na Tabela de Páginas quanto na TLB antes do novo quadro ser carregado do disco. A utilização do modelo de desenvolvimento *Spec-Driven Development* (SDD) em conjunto com o suporte de IA auxiliou na organização do pensamento lógico e na depuração de blocos específicos do código, sem, contudo, substituir a necessidade de um entendimento profundo sobre a arquitetura do gerenciador. Em suma, o projeto cumpriu seu papel de consolidar a ponte entre a teoria de Sistemas Operacionais e a prática de engenharia de software, evidenciando que o gerenciamento eficiente de memória é um pilar essencial para a performance e estabilidade de qualquer sistema computacional.

5. DOCUMENTAÇÃO DO USO DE INTELIGÊNCIA ARTIFICIAL

5.1 Metodologia (Spec-Driven Development - SDD)

Para o desenvolvimento deste projeto, a utilização de ferramentas de Inteligência Artificial (IA) seguiu estritamente o modelo de **Spec-Driven Development (SDD)**. Esta metodologia garante que a IA atue como uma extensão do entendimento técnico do grupo, fundamentando-se nos seguintes pilares:

- **Definição de Problema:** Antes de qualquer interação, o problema foi claramente delimitado (ex: a necessidade de implementar o algoritmo de *Aging*).
- **Especificação de Requisitos:** Os prompts incluíram os requisitos funcionais e as restrições impostas pelo projeto (ex: linguagem C, uso de bitwise, memória limitada).
- **Transparência:** Todas as interações estão listadas abaixo, com o objetivo de demonstrar a colaboração ética e o aprendizado contínuo.

5.2 Registro de Prompts e Interações

Fase do Projeto	Objetivo do Prompt	Prompt Utilizado	Contribuição da IA
Arquitetura	Definir estrutura de dados da Tabela de Páginas	"Atue como um engenheiro de software sénior. Defina a struct <code>page_table_entry_t</code> em linguagem C para um simulador de memória virtual. A estrutura deve conter os campos necessários..."	Auxiliou na modelagem de uma estrutura eficiente em conformidade com as restrições de controlo de estado persistente por página virtual.
Fundamentação e Teoria	Compreender os conceitos para o relatório	"o que é memória virtual" e "qual a teoria por trás"	Explicou de forma estruturada a abstração de memória, o processo de tradução via TLB/Tabela e a teoria por trás da paginação por procura (demand paging).
Implementação (main.c)	Entender e implementar extração de bits	"como vou fazer isso" seguido de "desligue o pato e seja direto" e "Complete o que falta [código main.c]"	Explicou a lógica de máscara e deslocamento (<code>>> 8</code> e <code>& 0xFF</code>) e gerou o código inicial para extração de página, offset e cálculo do endereço físico.

Tradução	Lógica de extração de bits para Página e Offset (Refinement o SDD)	"Explique como utilizar operações bitwise em C (shift e máscara) para extrair o número da página (8 bits mais significativos) e o offset (8 bits menos significativos)..."	Forneceu a sintaxe exata das operações de fatiamento implementadas no ciclo principal de tradução.
Implementação (main.c)	Corrigir <i>shadowing</i> e limpar fluxo principal	"[código main.c com redefinições de variáveis]"	Identificou redundâncias e limpou o código do fluxo principal para garantir que o <code>logical_address</code> não fosse sobrescrito.
Implementação (tlb.c)	Compreender e implementar a busca e FIFO na TLB	"Estou desenvolvendo um simulador de gerenciador de memória virtual... Inicialmente, preciso implementar os TODO's pela busca na TLB... Gostaria que você explicasse primeiro como ocorre uma consulta na TLB... e depois implementasse as funções..."	Detalhou a diferença entre TLB Hit e Miss e forneceu a implementação base das funções de procura (<code>tlb_lookup</code>) e inserção FIFO (<code>tlb_insert</code>).
Algoritmo (TLB)	Implementar a política de substituição FIFO (Refinement o SDD)	"Como devo estruturar a política de substituição FIFO (First-In, First-Out) num array estático que representa o TLB em C? Mostre a lógica	Orientou a criação da variável global e a lógica do índice circular (<code>fifo_next = (fifo_next + 1) % TLB_SIZE</code>) para

		para manter um índice circular..."	substituição eficiente na cache.
Implementação (memory.c)	Implementar tratamento de <i>Page Fault</i>	"[código memory.c com TODOs] agora complete esse"	Escreveu a lógica da função <code>handle_page_fault</code> , incluindo a procura no ficheiro <code>BACKING_STORE.bin</code> via <code>fseek/fread</code> e a invalidação de páginas vítimas.
I/O (Memória)	Leitura de bloco no <i>Backing Store</i> (Refinement o SDD)	"Forneça um exemplo em C de como ler um bloco específico de 256 bytes de um ficheiro binário ('BACKING_STORE.bin') utilizando acesso aleatório. Indique a sintaxe exata..."	Solucionou a mecânica de paginação sob demanda, definindo corretamente os deslocamentos do ponteiro para recuperar quadros do armazenamento secundário.
Implementação (page_table.c)	Implementar algoritmo de <i>Aging</i> (LRU)	"[código page_table.c com TODOs] complete esse"	Forneceu a implementação correta do deslocamento de bits (<code>>>= 1</code>) e inserção do bit de referência (
Algoritmo (Memória)	Algoritmo de <i>Aging</i> (Refinement o SDD)	"Escreva a lógica em C para a atualização dos contadores de envelhecimento (aging) das páginas na memória. A cada atualização, o	Forneceu a base algorítmica exata para o deslocamento de bits e a correta aplicação da máscara binária

		contador de 8 bits deve ser deslocado..."	para preservar o histórico temporal.
Implementação (statistics.c)	Implementar contadores de métricas de desempenho	"Agora, para a próxima parte do meu projeto, será necessário implementar os métodos de um arquivo statistics... Gostaria que você explicasse primeiro sobre a importância desses contadores... e em seguida implementasse os métodos..."	Explicou a relevância das métricas para a avaliação de desempenho (Hit/Miss Rate) e forneceu a implementação das funções de contagem de acessos, faltas de página e acertos na TLB.
Depuração	Resolver <i>Segmentation Fault</i>	"Estou a desenvolver um simulador de memória virtual em C e enfrento um erro de 'Segmentation fault' ao aceder à matriz <code>physical_memory</code> ... Identifique o motivo..."	Identificou a ausência de validação prévia no array de mapeamento <code>frame_to_page</code> e orientou a implementação de verificações de contorno estritas (0 a <code>NUM_FRAMES</code>).
Revisão e Validação	Confirmar a política de substituição adotada	"[Upload dos códigos] esse e o código guarde para responder o que eu pedir, qual foi? lru ou fifo"	Analisou o código consolidado e confirmou teoricamente que a implementação correspondia ao algoritmo de <i>Aging</i> (LRU aproximado) e não FIFO.

Redação do Relatório	Estruturar as secções textuais obrigatórias	"INTRODUÇÃO [...] faz", "Descrição do desenvolvimento [...] faz e deixe espaço", "Resultados obtidos [...] faz e deixe espaço"	Gerou os templates técnicos e académicos para as secções 1, 2, 3, 4 e 5 do relatório, deixando lacunas para personalização do grupo.
Redação do Relatório	Preencher as lacunas técnicas do relatório	"[Upload de prints das secções vazias do relatório (Decisões, Dificuldades, Conclusão)]"	Escreveu os parágrafos técnicos detalhando o uso de operações <i>bitwise</i> , a superação de <i>Segmentation Faults</i> e a consistência de integração TLB/Tabela de Páginas.
Testes	Estruturar a análise de resultados	"Preciso de redigir a secção de resultados do meu relatório técnico. Como posso estruturar uma análise comparativa de performance..."	Orientou a criação da metodologia de comparação quantitativa (Tabela 1) e fundamentou a interpretação teórica sobre o impacto da localidade de referência no desempenho do sistema.

5.3 REPOSITÓRIO DO TRABALHO

https://github.com/rodrigoggmagalhaes/so_gerenciadormemoriavirtual